
ユーザ・アプリケーション作成の基礎

演習パッケージ: P05_FirstStep

Geant4 11.3準拠

Geant4講習会資料: HEP/NP/Space 2025



KEK

大学共同利用機関法人
高エネルギー加速器研究機構

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

演習トピックス

[概要] P05_FirstStep/WaterBoxプログラムを使い、mainプログラムの構造の理解と実習

[目次]

1. 演習の準備

- Hands-on全体構成と進め方、C++コードのディレクトリ構造

2. P05_FirstStep/WaterBoxプログラムを実行してみる

- 簡単なアプリをビルドし、実行モジュールを作る方法を学ぶ

3. マクロファイルを使う

- マクロファイルの基本的な使い方を学ぶ

4. マクロコマンドで実行条件を変えWaterBoxを走らせる

- マクロコマンドで入射粒子の条件を変える

5. P05_FirstStep/ WaterBoxのmainプログラム構造

- P05_FirstStep/WaterBox のmainプログラムの構造を理解する

自習課題： 以下の課題は演習で扱わない。演習終了後、各自で実行してみる

- マクロファイルを使ったBatch処理の実行
- “.g4session”を使ったユーザ・インターフェイスの選択

演習の準備

演習トピックス

[概要] [P05_FirstStep/WaterBox](#) プログラムを使い、[mainプログラム](#)の構造の理解と実習

[目次]



1. 演習の準備
 - Hands-on全体構成と進め方、C++コードのディレクトリ構造
2. [P05_FirstStep/WaterBox](#)プログラムを実行してみる
 - 簡単なアプリをビルドし、実行モジュールを作る方法を学ぶ
3. マクロ ファイルを使う
 - マクロファイルの基本的な使い方を学ぶ
4. マクロ コマンドで実行条件を変え [WaterBox](#)を走らせる
 - マクロコマンドで入射粒子の条件を変える
5. [P05_FirstStep/ WaterBox](#)のmainプログラム構造
 - [P05_FirstStep/WaterBox](#) のmainプログラムの構造を理解する

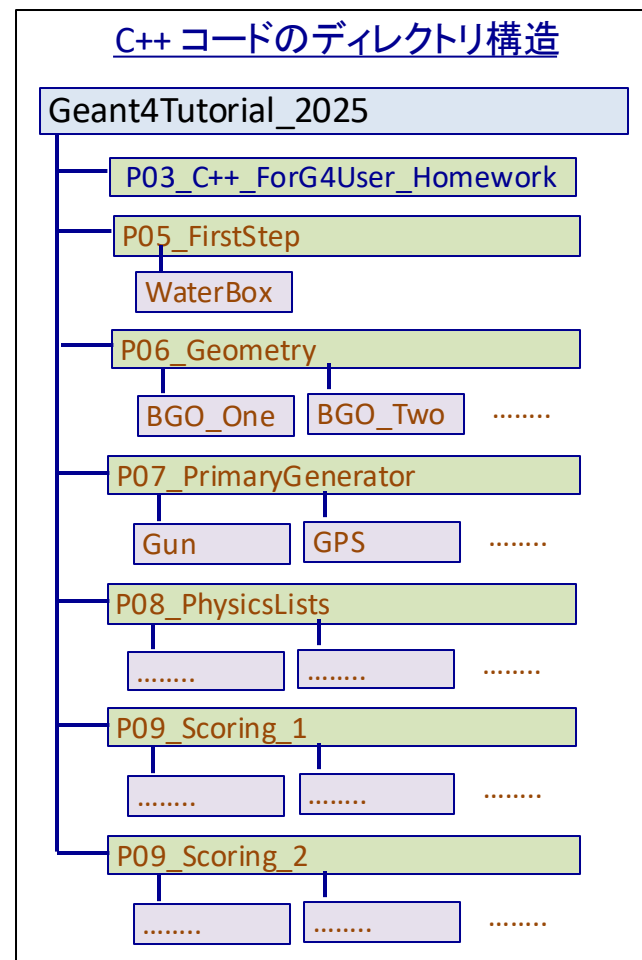
Hands-on演習項目と進め方、C++コードのディレクトリ構造

■ Hands-onの演習項目及びコード・ディレクトリ

1. ユーザ・アプリケーション作成の基礎 (P05_FirstStep)
 - ユーザ作成するmainプログラム構造の理解と実習
2. ジオメトリの組み込み (P06_Geometry)
 - ユーザ作成必須クラス1: ジオメトリ作成の実習
3. 初期粒子の発生方法 (P07_PrimaryGenerator)
 - ユーザ作成必須クラス2: 初期粒子発生の実習
4. Physics Listの使用方法 (P08_PhysicsLists)
 - ユーザ作成必須クラス3: physics list作成の実習
5. 結果の取得と出力 その1 (P09_Scoring_1)
 - カロリメータを使った物理観測量の取得法の実習
6. 結果の取得と出力 その2 (P010_Scoring_2)
 - 位置検出器を使った物理観測量の取得法の実習

■ Hands-onの進め方

1. 各演習開始時にディレクトリの一つ(例:P05_FirstStep)に移動
2. 各ディレクトリは演習トピックスに対応したサブ・ディレクトリで構成
3. サブ・ディレクトリの構造は全ての演習で同一 (次スライドを参照)
4. 演習はトピックスごとのサブ・ディレクトリに移行して進める (例:P05_FirstStep/WaterBox)



演習プログラムのサブ・ディレクトリ構造の確認

課題:1 演習サブ・ディレクトリ構造は全演習を通し同一: P05_FirstStep/WaterBoxを試みる

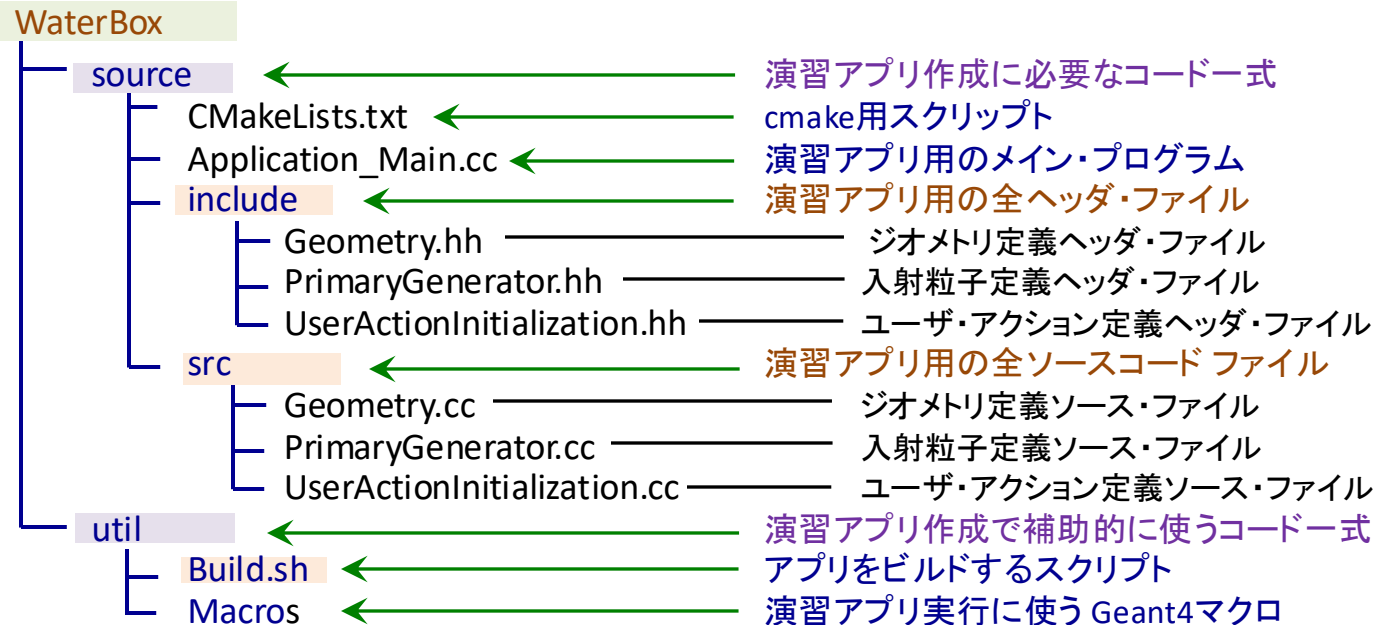
1) 以下に従って、'ls'コマンドを実行

```
$ cd ~/Geant4Tutorial_2025
$ ls P05_FirstStep/WaterBox
```

[メモ]

1. コマンド入力には「tabキー」補完機能を使う
2. スライドのコマンドを「コピペ」するのは危険

2) P05_FirstStep/WaterBoxのファイル構造が下記であることを確認



[注 1] Geant4のライブラリは以下にインストールされている

/opt/geant4/11.3.0/lib64/

[注 2] Geant4アプリケーション実行時に必要な物理過程反応断面積データへのパスはユーザの".bashrc"で自動的に環境変数へ設定済み

source /opt/geant4/11.3.0/bin/geant4.sh

P05_FirstStep/WaterBoxプログラムを実行してみる

演習トピックス

[概要] P05_FirstStep/[WaterBox](#)プログラムを使い、[mainプログラム](#)の構造の理解と実習

[目次]

1. 演習の準備

- Hands-on全体構成と進め方、C++コードのディレクトリ構造

2. P05_FirstStep/[WaterBox](#)プログラムを実行してみる

- 簡単なアプリをビルドし、実行モジュールを作る方法を学ぶ

3. マクロ ファイルを使う

- マクロファイルの基本的な使い方を学ぶ

4. マクロ コマンドで実行条件を変え[WaterBox](#)を走らせる

- マクロコマンドで入射粒子の条件を変える

5. P05_FirstStep/ [WaterBox](#)の[mainプログラム](#)構造

- P05_FirstStep/[WaterBox](#) のmainプログラムの構造を理解する

P05_FirstStep/WaterBox プログラムの概要

■ 演習プログラムの目的

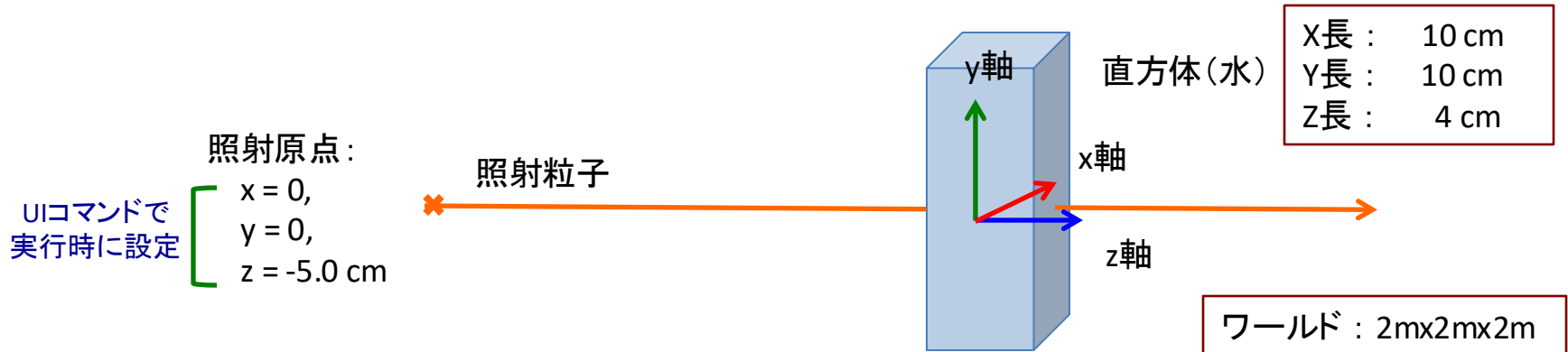
- 簡単なアプリケーションをビルドし、実行モジュールを作る方法を学ぶ

■ プログラムの構成

- mainプログラム: 最も単純な対話型アプリケーション
- ジオメトリ: 直方体(物質は水)が一つだけ置かれている
- PhysicsList: 高エネルギー領域で汎用的な‘FTFP_BERT’とよばれるもの
- PrimaryGenerator: 標準ツールとして提供される‘ParticleGun’とよばれるもの

■ 組み込まれているジオメトリと粒子発生機能

- 粒子を一つだけ直方体に向けて照射
- デフォルトでは発生粒子はgeantino



P05_FirstStep/WaterBoxのビルド

課題: 1 WaterBoxをビルドして実行ファイルを作成する

1) P05_FirstStep/WaterBoxのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P05_FirstStep/WaterBox
$ ls
source/  util/
```

[メモ]

1. コマンド入力は「tabキー」補完機能を使う
2. スライドのコマンドを「コピー」は危険

2) ビルドするための作業ディレクトリを作成

```
$ mkdir build
$ ls
build/  source/  util/
```

ビルド作業のディレクトリ名は習慣に従いbuildとする

3) buildディレクトリでビルドを実行 [注]

```
$ cd build
$ cmake ../source
$ make
$ make install
```

sourceにあるCMakeLists.txtを使いcmakeを実行

自動生成されたMakefileを実行 (-jオプション使用可能)

binディレクトリが自動作成され実行ファイルを収納

- makeでbuild内に実行ファイルが作られ、make installでそれがbinにコピーされる (初めの makeは省略可能)
- cmake実行でbuildディレクトリの中に多くのファイルが自動作成されるが、その内容は無視してOK

4) ビルド実行後のディレクトリ構造確認

```
$ cd ..
$ ls
bin/  build/  source/  util/
$ ls bin
Application_Main*
```

CBDirディレクトリへ移動

binディレクトリが自動生成

実行ファイルが生成

[注] 実行ファイルが上手く作れない場合

ステップ1)に戻り、以下のスクリプトを実行すれば、実行ファイルが自動生成

./util/Build/Build.sh

P05_FirstStep/WaterBox: Application_Mainの実行

課題:2 Application_Main を実行する

1) アプリ実行を行う作業ディレクトリを新たに作成する

```
$ pwd
...../P05_FirstStep/Waterbox
$ mkdir TestBench
$ ls
TestBench/  bin/    build/  source/  util/
$ cd TestBench
$ ../bin/Application_Main
```

現在のディレクトリがWaterBoxであることを確認:
そうでなければ移動する

アプリを実行するための作業ディレクトリを作る
[注] 名前は自由だが、全演習を通してこの名前を使う

TestBenchディレクトリでApplication_Main実行
[注] アプリ実行の際、コマンドライン引数が無いと
対話モード(interactive mode)が起動される

2) 新しいウィンドが開き、次スライドで示す標準ユーザインターフェイス(Qt画面)が出力されることを確認

3) アプリ実行に使った端末ウィンドに以下のメッセージが出力されることを確認

```
*****
Geant4 version Name: geant4-11-03 [MT] (6-December-2024)
Copyright : Geant4 Collaboration
References : NIM A 506 (2003), 250-303
             : IEEE-TNS 53 (2006), 270-278
             : NIM A 835 (2016), 186-225
WWW : http://geant4.org/
*****

<<< Geant4 Physics List simulation engine: FTFP_BER
```

← 使用しているGeant4の
バージョン情報

← 使用するPhysics Listの名前

以下省略

初期Qtウィンドウ

課題:3 Qtウィンドウの全体構成を学ぶ

- アプリが対話モードで起動されたので、Geant4標準ユーザインターフェイスのQt画面が表示される
- アプリはジオメトリ、物理過程設定などの初期設定が終了して、ユーザの**コマンド待ち(idle)状態**になっている
- 下に示す**ユーザ・コマンド入力ウインド(窓)**に任意のコマンドを入れることでアプリを対話的に動作させる

コマンドアイコン

- よく使うコマンドをアイコン化
- 使用Versionで表示が異なる

help、ジオメトリ情報、コマンド履歴等を表示の選択

- 現時点ではHelpが選択されているので、画面下にユーザ・コマンド一覧が表示されている

Visualizationを起動すると、可視化画面がここに表示される

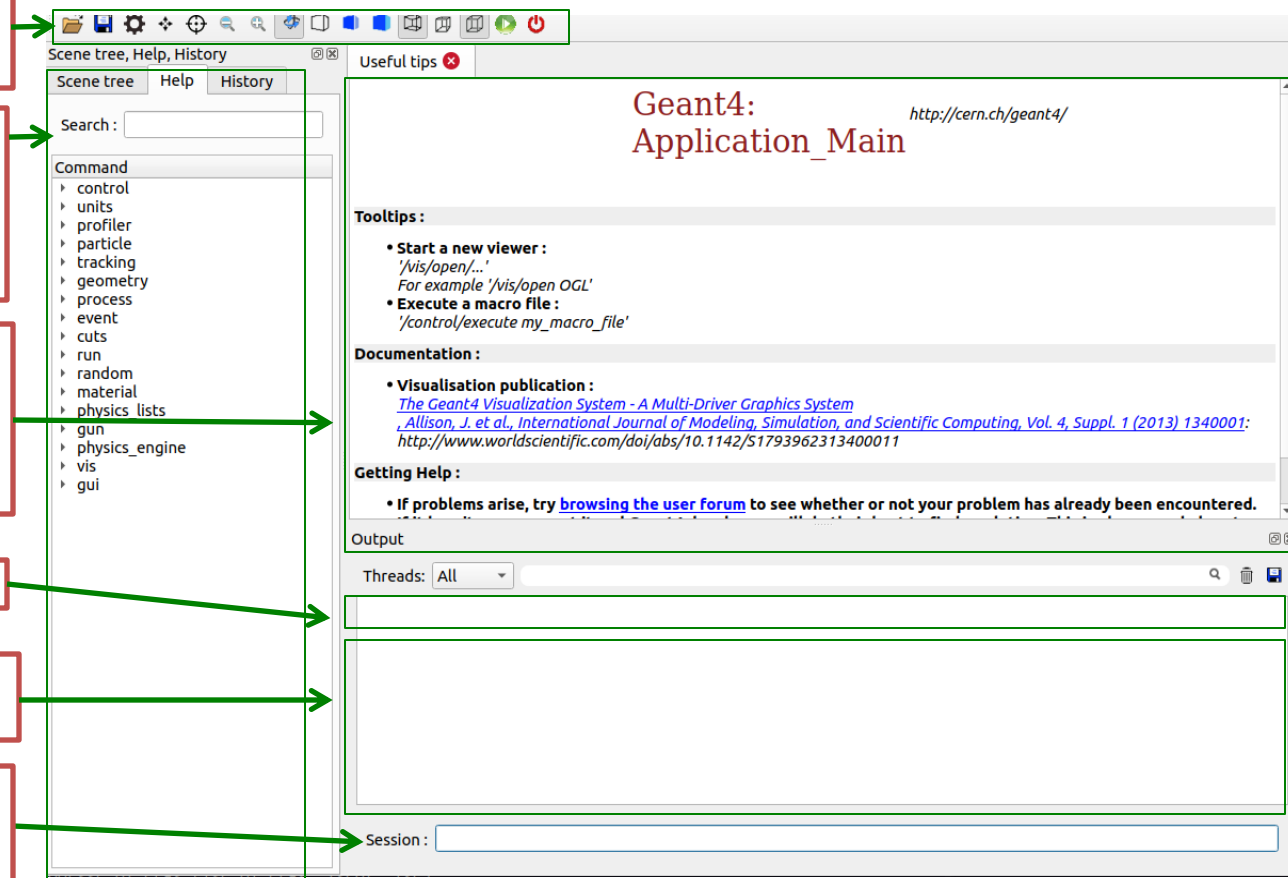
- 現時点では起動していないのでQtからの初期メッセージが表示されている

下記の出力内容の検索

アプリのG4coutからの出力内容の表示

ユーザ・コマンドの入力窓

- ここにコマンドを入れることでアプリの動作をコントロールする

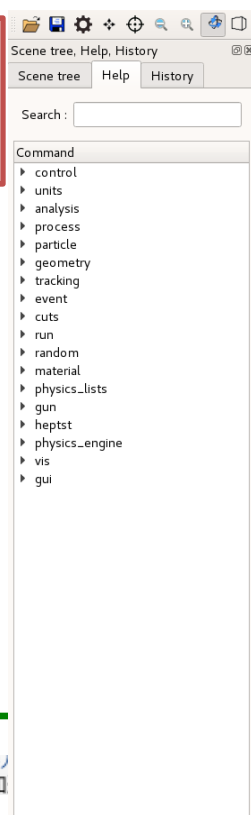


ユーザ・コマンド

課題:4 Geant4が標準で用意するユーザ・コマンド(User Interface Commands)を概観する

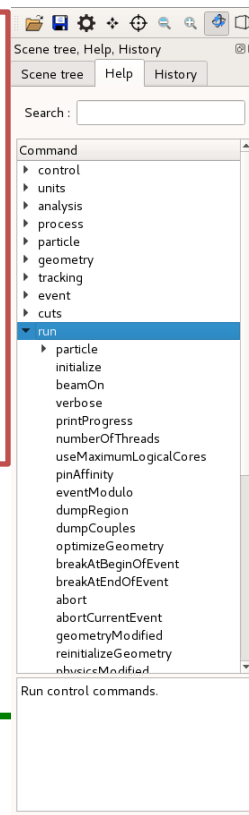
- コマンドとはアプリの実行を対話的に制御するために用いるもので、**UI コマンド**ともよばれる
- 用意されているコマンドは多岐にわたるが、ユーザが知っておくべきものは限られている
 - ➡ この講習会で出てくるコマンドを知るのでもずはOK
- 対話的コマンド入力は間違いやすいので、実際はコマンドの**マクロファイル**を用意して実行する
 - ➡ **マクロファイル**の導入は先の課題で行う
- 対話的だけでなく、アプリをバッチ的に処理する際もアプリ制御に**マクロファイル**を使用する（**自習課題参照**）
- ユーザが使用可能なUIコマンドに関する情報はQtの左ウィンドウで見ることができる:

Qt左ウィンドウのmenuで
'Help'を選択することで、
UIコマンドの一覧が表示
される

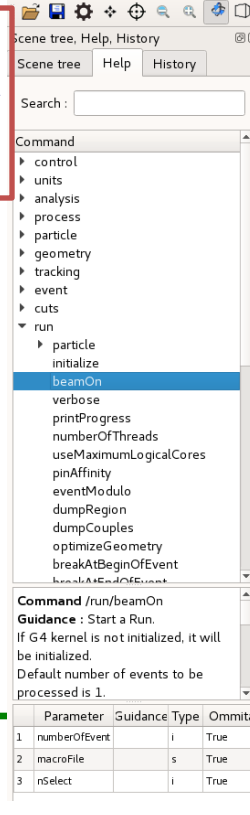


左図のコマンドの一覧で
'Run'を選択すると'Run'
に属するコマンドの一覧
が表示される

[注]
各行の先頭に三角印
があるのコマンド、無い
のはフォルダで、その
中にさらにコマンドが
記述されている



'Run'に属するコマンドの
一覧の中で'beamOn'を
選択すると、このコマンド
の機能と引数情報が下
に表示される



粒子のtracking詳細情報を書き出す

課題:5 粒子のtracking詳細情報の取得法を知る

- Geant4シミュレーションがどの様に進行するかを理解するには、粒子trackingのステップ情報を逐次出力してみることである ← geant4は粒子をstepを単位としてtrackingする
- この課題では、geantino粒子を与えられている水の直方体に入射して、そのステップ情報を出力する

1) まず、gunコマンドをQtのコマンド窓に入力して、geantinoの発生点(x,y,z)と照射方向の方向余弦(dx,dy,dz)を設定

```
/gun/position 0.0 0.0 -5.0 cm  
/gun/direction 0.0 0.0 +1.0
```

[注] コマンド補完機能が使えるので
コマンド名を全てタイプしなくてもOK

[注] gunはdefaultで粒子をgeantinoにする

2) 続いて、以下の粒子照射コマンドを入力し、geantinoを1回照射する

```
/run/beamOn 1
```

- 初めて/run/beamOnを実行すると、アプリの出力窓に粒子相互作用の設定条件が必ず出力される

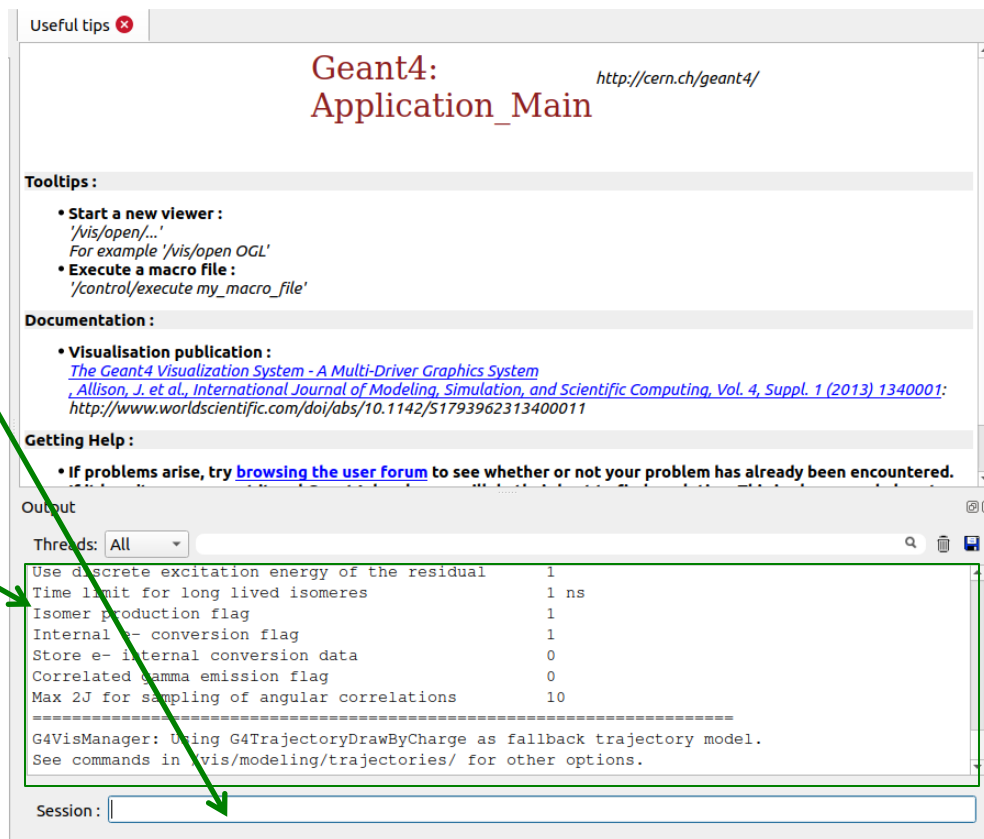
[注] 出力情報の内容は当面無視してOK

- しかし、geantinoのtracking過程などのシミュレーション情報は何も出力されない

3) さらに、5回のgeantinoを照射してみる

```
/run/beamOn 5
```

- やはり、何の情報も出力されない



粒子のtracking詳細情報を書き出す — つづき

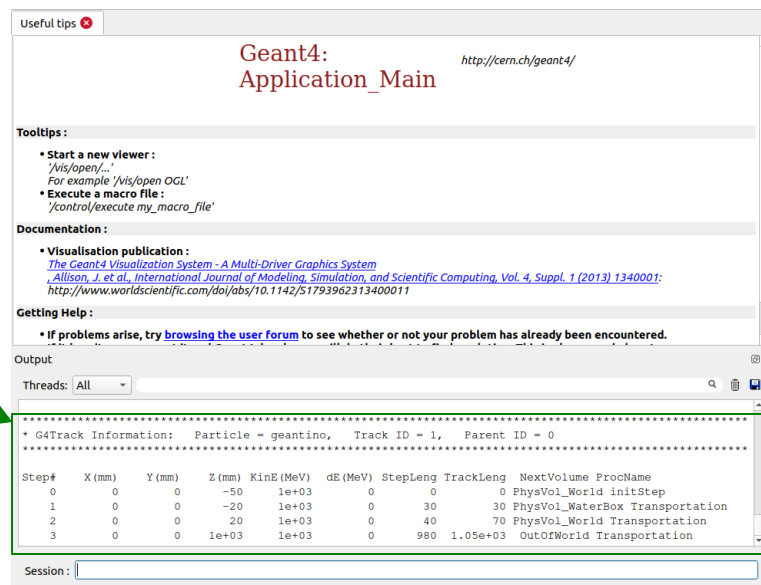
課題:5 粒子のtracking詳細情報の取得法を知る — つづき

4) 今度は以下のコマンドを入力してみる

```
/tracking/verbose 1  
/run/beamOn 1
```

- アプリ出力窓にgeantinoのstep情報が出力される
- /tracking/verboseコマンドはtracking情報の出力をコントロールするもの
- 前ステップでgeantinoは正しくtrackingされていたのだが、/tracking/verboseがセットされていなかったので出力が何もなかった

[注] /tracking/verbose の引数値で出力詳細レベルをコントロールできる



5) 上で得られたgeantinoのstep情報部分の内容を拡大すると以下ようになる

```
*****  
* G4Track Information: Particle = geantino, Track ID = 1, Parent ID = 0  
*****  
  
Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName  
0         0         0        -50     1e+03      0         0         0 PhysVol_World initStep  
1         0         0        -20     1e+03      0         30        30 PhysVol_WaterBox Transportation  
2         0         0         20     1e+03      0         40        70 PhysVol_World Transportation  
3         0         0       1e+03     1e+03      0        980     1.05e+03 OutOfWorld Transportation
```

trackingの
各ステップの
情報

- Geant4は粒子をstep単位に進ませる。構造物の境界では必ず一つのstepが終了する
- geantinoは物質と相互作用はしないので、step終了点は直方体(水)およびワールドの境界でのみで生じている

粒子のtracking過程をグラフィック表示する

課題:6 粒子trackをグラフィックを用いて可視化する方法を知る

- シミュレーション開発では粒子trackをグラフィックで見ることが不可欠
- 先の課題で扱ったgeantino粒子のtrackingを可視化する

1) まず、前課題の終了時点でのQtコマンド窓に以下のコマンドを入力してグラフィック機能を起動

```
/vis/open OGL
```

- Qt上部画面がグラフィック画面(黒地)になることを確認
[注] OGLを指定することでGeant4の標準グラフィック機能が起動する - 通常はこれを使うのでOK

2) worldボリュウムのboxを表示

```
/vis/drawVolume world
```

- 表示画面をマウスでクリックした後、ドラッグしてboxを回転させることができることを確認
- boxが斜め上から見える位置にする

3) 粒子飛跡を可視化させる条件設定

```
/vis/scene/add/trajectories
```

4) geantinoを1回照射する

```
/run/beamOn 1
```

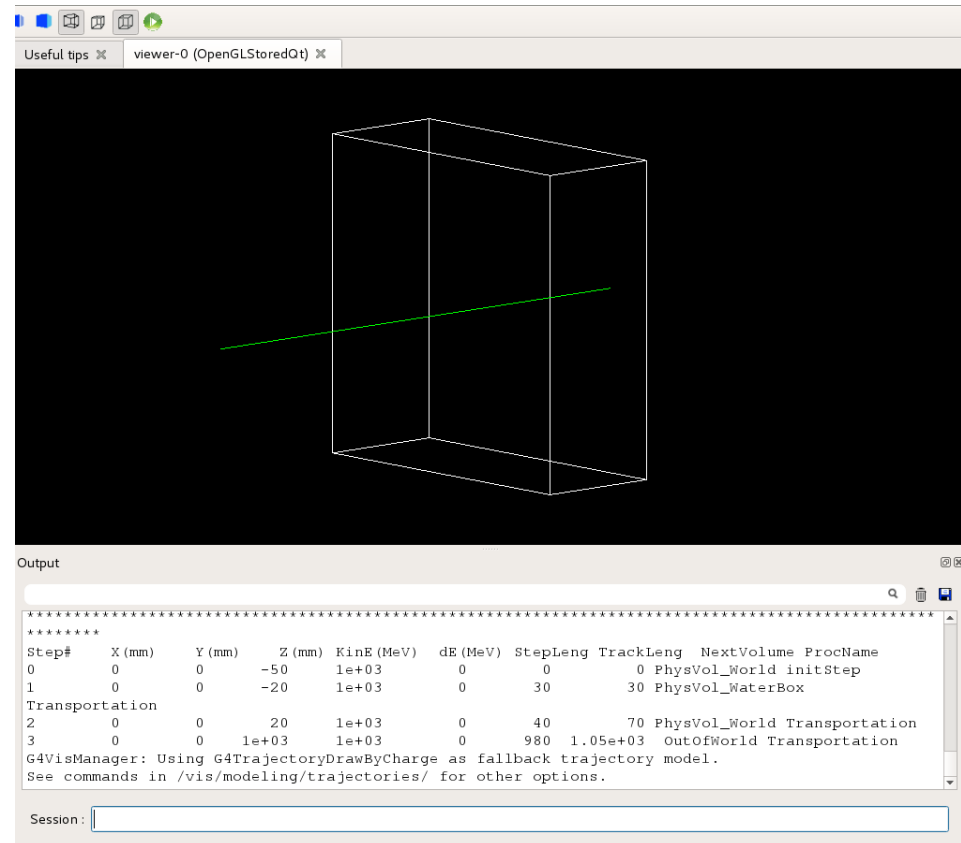
- 右図の様にgeantinoの飛跡が表示されることを確認

5) さらにgeantinoを数回照射した後、アプリを終了

```
/run/beamOn 15  
exit
```

対話セッションを閉じてアプリを終了するコマンド

[注] 'exit' は先頭に '/' が不要



マクロ ファイルを使う

演習トピックス

[概要] [P05_FirstStep/WaterBox](#) プログラムを使い、[mainプログラム](#)の構造の理解と実習

[目次]

1. 演習の準備

- Hands-on全体構成と進め方、C++コードのディレクトリ構造

2. [P05_FirstStep/WaterBox](#)プログラムを実行してみる

- 簡単なアプリをビルドし、実行モジュールを作る方法を学ぶ

3. マクロ ファイルを使う

- マクロファイルの基本的な使い方を学ぶ

4. マクロ コマンドで実行条件を変え [WaterBox](#)を走らせる

- マクロコマンドで入射粒子の条件を変える

5. [P05_FirstStep/ WaterBox](#)の[mainプログラム](#)構造

- [P05_FirstStep/WaterBox](#) のmainプログラムの構造を理解する

マクロファイルを使ってみる

課題:1 マクロファイルの基本的な使い方を学ぶ

- 前の章ではUIコマンドを一つ一つ手で入力したが、これは間違いやすいし手間が大変
- Geant4はUIコマンドを一括して実行できるマクロ機能を用意
- シミュレーションを実行には、マクロファイルを作成、実行するのが標準手法
- 以下では、先の課題で手入力したコマンドが書かれたマクロファイルを用い、それを実行してみる

1) TestBenchディレクトリに演習用に用意されているマクロファイルをコピー

```
$ pwd ← 前課題の終了時点で、以下のdirにいるはず：もし異なる場合、下記dirへ移動
/home/g4user/Geant4Tutorial_2025/P05_FirstStep/WaterBox/TestBench
$ cp ../util/Macros/myFirstMacro.mac .
$ less myFirstMacro.mac ← マクロの内容を確認
```

```
## My first Geant4 macro file
```

```
/gun/position 0.0 0.0 -5.0 cm
/gun/direction 0.0 0.0 +1.0
/vis/open OGL
/vis/drawVolume world
/vis/scene/add/trajectories
/tracking/verbose 1
/run/beamOn 1
```

先頭が"#"で始まるのは
コメントライン

マクロファイルに先の課題で
タイプしたコマンドが書かれ
ていることを確認

マウスの右クリックで表示されるメニュー
からview条件を変更可能

- 移動、回転、拡大
 - 構造体のsurface表示など
- また、左クリック、wheelも使用可能

2) TestBenchディレクトリでアプリケーションを再実行

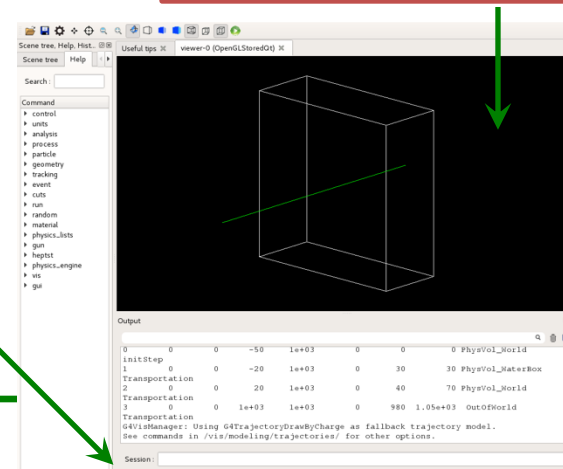
```
$ ../bin/Application_Main
```

3) Qtコマンド窓に以下を入力しマクロファイルを実行

```
$ /control/execute myFirstMacro.mac
```

- 前の課題で見たQt窓と全く同じであることを確認

4) Qtコマンド窓に'exit'コマンドを入れ、アプリを終了



Hands-onで使用するマクロ ファイル

課題:2 以後のHands-onで使用するマクロファイルの構成を理解する

- 前課題で使った'myFirstMacro.mac'ではコマンドは羅列的に記述
- 本Hands-onでは以下で述べる階層構造マクロを標準として使用

[注]

ユーザは自分のアプリで
この構造を踏襲する必要なし

1) Hands-onで今後使うマクロファイルをコピーする

```
$ pwd  
$ cp ../util/Macros/* .  
cp: overwrite './myFirstMacro.mac'? y/n  
$ ls  
GlobalSetup.mac myFirstMacro.mac primaryGeneratorSetup.mac visSetup_Simplex.mac verboseSetup.mac
```

このマクロは使用しない。これ以外が今後使用する階層構造マクロ

2) GlobalSetup.macの内容をみる

```
$ less GlobalSetup.mac
```

```
#####  
# GlobalSetup.mac: Top level macro to set up the initial environment to run "appli"  
# [Note] Geant4 Tutorial for Hep/Space Users  
#####  
  
## Set up primary generaotr  
/control/execute primaryGeneratorSetup.mac  
  
## Set up visulalization ennriornment  
/control/execute visSetup_Simplex.mac  
  
## Set up verbosity  
/control/execute verboseSetup.mac  
  
## Invoke the application  
/run/beamOn 0      # arg = 0; Only invokes initialization - no event generation  
                  # arg > 0; Specified number of events will be executed
```

GlobalSetup.mac
マクロファイル実行のルート

以下の3つのマクロを実行

primaryGeneratorSetup.mac
照射粒子のキネマテックス等を
設定するマクロ

visSetup_Simplex.mac
可視化環境の設定マクロ

verboseSetup.mac
Geant4カーネルからの情報出
力を制御するマクロ

マクロ実行で粒子照射を開始
したければ、照射回数を指定

Hands-onで使用するマクロ ファイル — つづき

課題:3 GlobalSetup.macが実行する3つのマクロ ファイルの内容を見る

1) primaryGeneratorSetup.macの内容

```
## Particle type
/gun/particle geantino

## Kinematical information
/gun/energy 1.0 MeV
/gun/position 0.0 0.0 -5.0 cm
/gun/direction 0.0 0.0 +1.0
```

照射粒子のキネマテックス等を設定するマクロ

- 粒子をgeantinoと指定
- 照射位置と方向を指定
- 粒子のEnergy指定
(geantinoでは意味はない)

2) visSetup_Simplex.macの内容

```
## Specify the viewer
/vis/open OGL                                # Use OpenGL

## Specify objects to draw
/vis/drawVolume world                        # Volumes
/vis/scene/add/trajectories smooth           # Particle trajectories
/vis/scene/endOfEventAction accumulate      # Superimpose all of the events

## Add decorations
/vis/scene/add/axes                          # Axes (x=red, y=green, z=blue)

## To get nice view
/vis/viewer/set/style wireframe              # Volume style (surface/wireframe)
/vis/viewer/set/auxiliaryEdge true           # Show edges of curved surfaces
/vis/viewer/set/lineSegmentsPerCircle 100   # Make circles smoothed
/vis/viewer/set/viewpointThetaPhi 120 150    # Nice camera angle
```

可視化環境の設定マクロ

- OpenGLを使うことを指定
- 描画する内容を指定
- 描画視点を指定

[注] ここでは、最低限の環境設定のみがされている

3) verboseSetup.macの内容

```
## Commonly used verbosity
/run/verbose 1
/tracking/verbose 1

## Miscellaneous verbosity
/control/verbose 0
/event/verbose 0
/material/verbose 0
/process/verbose 0
/vis/verbose 0
```

Geant4カーネルからの情報出力を制御するマクロ

- Run情報の出力レベルを1に設定
- Tracking情報のレベルを1に設定
- 他の情報レベルは0に設定

Hands-on用マクロ ファイルを使ってみる

課題:4 Hands-on用のマクロファイルを使いアプリを走らせてみる

1) TestBenchディレクトリでアプリケーションを再実行

```
$ ../bin/Application_Main
```

2) Qtコマンド窓に以下のコマンドをタイプしてGlobalマクロを実行する

```
/control/execute GlobalSetup.mac
```

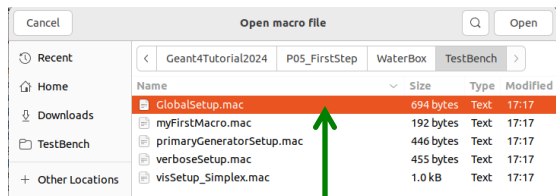
3) 続いて以下のコマンドをタイプして前演習での結果が再現していることを確認

```
/run/beamOn 1
```

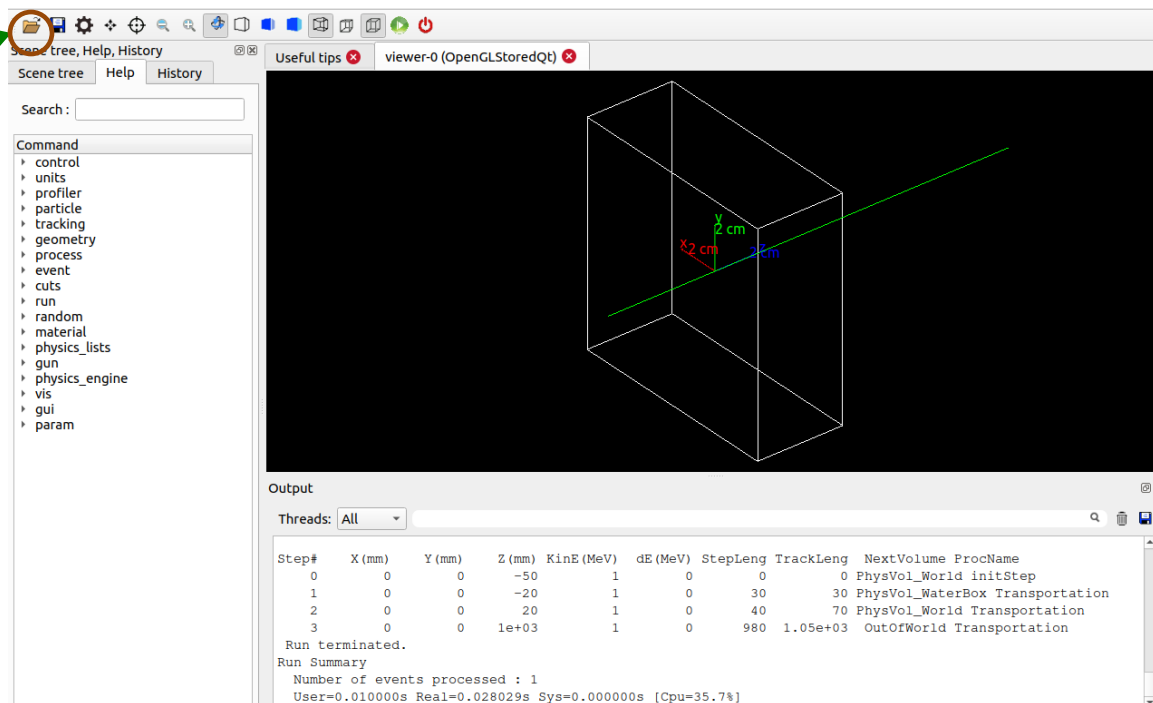
[Note]

ファイル・アイコンをクリックすることで、
上記2)のステップを省略してマクロ
実行可能

アイコンのクリックで以下の窓が開くので、
GlobalSetup.macをダブルクリック
すればOK:



ダブルクリック



マクロ コマンドで実行条件を変え WaterBoxを走らせる

演習トピックス

[概要] P05_FirstStep/[WaterBox](#)プログラムを使い、[mainプログラム](#)の構造の理解と実習

[目次]

1. 演習の準備

- Hands-on全体構成と進め方、C++コードのディレクトリ構造

2. P05_FirstStep/[WaterBox](#)プログラムを実行してみる

- 簡単なアプリをビルドし、実行モジュールを作る方法を学ぶ

3. マクロ ファイルを使う

- マクロファイルの基本的な使い方を学ぶ

4. マクロ コマンドで実行条件を変え [WaterBox](#)を走らせる

- マクロコマンドで入射粒子の条件を変える

5. P05_FirstStep/ [WaterBox](#)の[mainプログラム](#)構造

- P05_FirstStep/[WaterBox](#) のmainプログラムの構造を理解する

入射粒子をprotonに変えて照射してみる

課題:1 protonを一回、照射する

1) アプリを起動してGlobalSetup.macを実行する [注] 前の課題でアプリを実行したままであれば、そのまま次に進んでOK

2) 以下のコマンドを入力すれば、粒子をprotonに指定して、そのenergyを60MeVに設定できる

```
/gun/particle proton  
/gun/energy 60 MeV
```

3) protonを照射してみる

```
/run/beamOn 1
```

- 右図の様にprotonの飛跡が見えることを確認

[注] 上述の二種類のコマンド入力手法の両方を試してみること

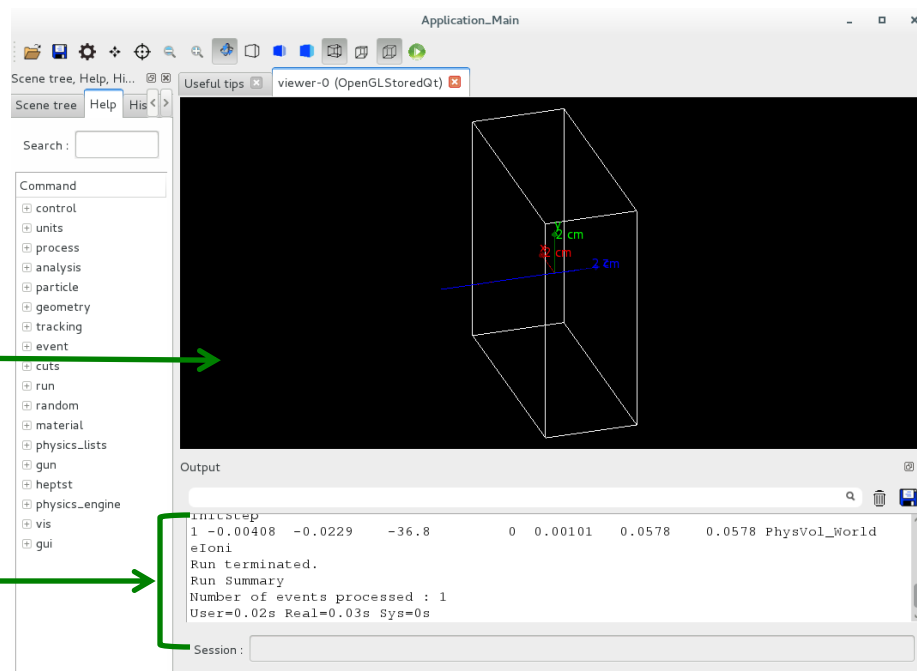
粒子飛跡の標準色表示は以下の通り

青: + 電荷の粒子

赤: - 電荷の粒子

緑: 0 電荷の粒子

proton及び物質との相互作用で作られた二次粒子のステップ情報が出力されている



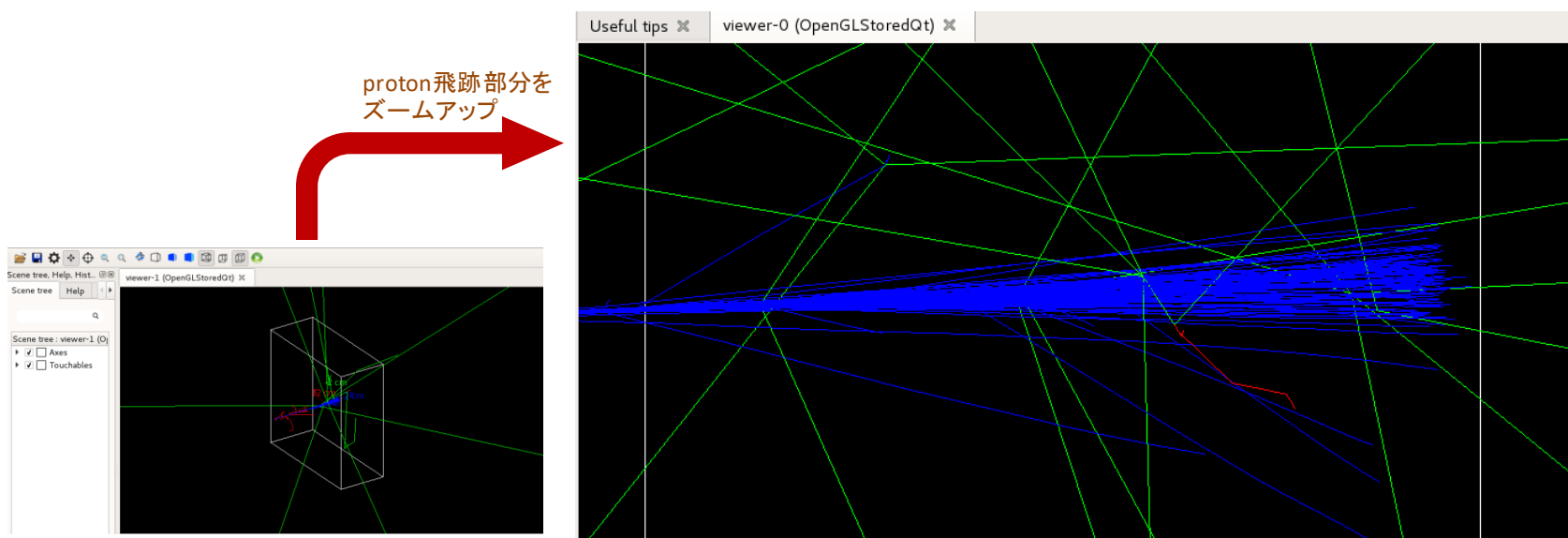
Protonを複数照射してrange strugglingをみる

課題:2 protonを100回、照射してrange strugglingを見てみる

1) 前の課題を終了した状態でprotonを100回照射

```
/run/beamOn 100
```

2) 表示画面でマウスを使い、proton飛跡を拡大・移動・回転し、range strugglingを目視してみる



粒子の種類、kinematical条件を変え照射してみる

課題:3 electronを照射してみる

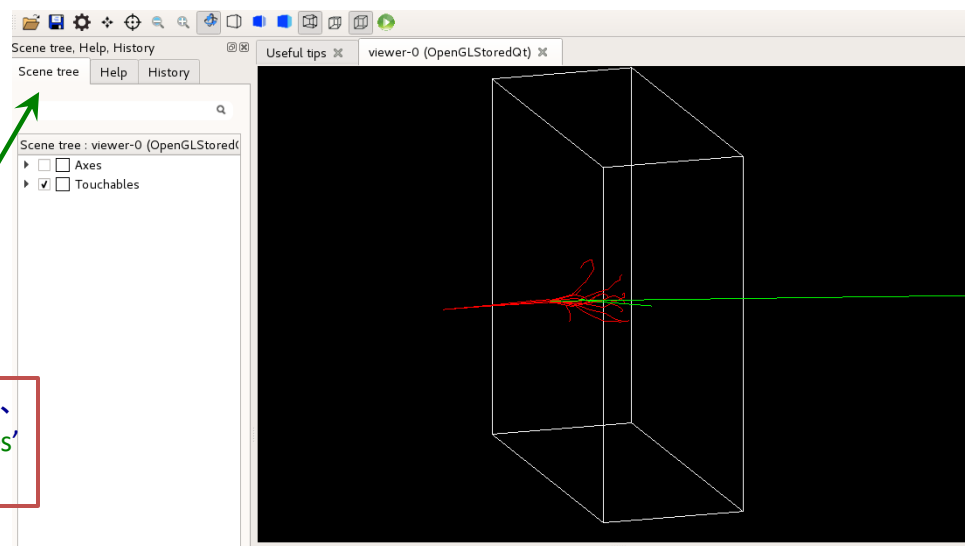
1) 例えば、以下の照射条件を設定する

```
/gun/particle e-  
/gun/energy 5 MeV
```

2) Electronを10回照射する

```
/run/beamOn 10
```

[注] 座標軸表示が邪魔な場合、
'Scene tree' メニューで 'Axes'
のチェックを外す



課題:4 その他、粒子の種類、kinematical条件を変え、粒子飛跡の振る舞いを観察してみる

[注] 作業が終了したら、Qtコマンド窓に 'exit' コマンドを入れ、アプリを終了

P05_FirstStep/WaterBoxのmainプログラム構造

演習トピックス

[概要] P05_FirstStep/[WaterBox](#)プログラムを使い、[mainプログラム](#)の構造の理解と実習

[目次]

1. 演習の準備

- Hands-on全体構成と進め方、C++コードのディレクトリ構造

2. P05_FirstStep/[WaterBox](#)プログラムを実行してみる

- 簡単なアプリをビルドし、実行モジュールを作る方法を学ぶ

3. マクロ ファイルを使う

- マクロファイルの基本的な使い方を学ぶ

4. マクロ コマンドで実行条件を変え[WaterBox](#)を走らせる

- マクロコマンドで入射粒子の条件を変える

5. P05_FirstStep/ [WaterBox](#)のmainプログラム構造

- P05_FirstStep/[WaterBox](#) のmainプログラムの構造を理解する



P05_FirstStep/WaterBox: mainプログラムの構造

課題: 1 mainプログラムの基本構造を知る `$ less waterBox/source/Application_Main.cc`

```
//+++++
// Geant4 Application: Tutorial course for Hep/Medicine Users
//+++++
#include "Geometry.hh"
#include "UserActionInitialization.hh"

#include "G4RunManager.hh"
#include "G4UImanager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"
#include "FTFP_BERT.hh"

//-----
int main( int argc, char** argv )
//-----
{
  // Construct a run manager
  auto* runManager = new G4RunManager();

  // Set up mandatory user initialization: Geometry
  runManager->SetUserInitialization( new Geometry{} );

  // Set up mandatory user initialization: Physics-List
  runManager->SetUserInitialization( new FTFP_BERT{} );

  // Set up user initialization: User Actions
  runManager->SetUserInitialization( new UserActionInitialization{} );

  // Initialize G4 kernel
  runManager->Initialize();

  // Create visualization environment
  auto visManager = new G4VisExecutive();
  visManager->Initialize();

  // Get UI (User Interface) manager
  G4UImanager* uiManager = G4UImanager::GetUIpointer();

  // Switch batch or interactive mode
  if ( argc == 1 ) { // Interactive mode - no command argument
    auto uiExec = new G4UIExecutive( argc, argv );
    uiExec->SessionStart();
    delete uiExec;
  } else { // Batch mode - 1st command argument is a macro-filename
    G4String macroName = argv[1];
    uiManager->ApplyCommand( "/control/execute " + macroName );
  }

  // Job termination
  delete visManager;
  delete runManager;

  return 0;
}
```

mainプログラムで使用するクラスの
ヘッダーファイルを全てincludeする

RunManagerを作る

ジオメトリの登録

- ユーザが定義するクラス

粒子と相互作用モデルを登録

- ツールキット提供クラス

primary generatorはUserActionを通
して登録

- ユーザが定義するクラス

以上の設定で初期化実行

アプリ実行時に引数がないなら
対話型セッションが起動

アプリ実行時にマクロファイル名
を引数として与えると、そのマクロ
内容をバッチ処理

アプリケーション終了処理

演習 終了

自習課題

マクロファイルを使ったBatch処理の実行

課題:1 バッチ処理でApplication_Mainを実行する

- 今までの課題では、アプリを対話型で処理してきた
- Application_Mainは実行時の引数にマクロファイルを指定することで、バッチ処理が可能である

1) TestBenchディレクトリに移動

2) TestBenchにすでに存在する'GlobalSetup.mac'をバッチ用のマクロファイルとして使うので、このマクロを以下のように編集する

```
## Set up primary generaotr
/control/execute primaryGeneratorSetup.mac

## Set up visulalization ennrionment
#/control/execute visSetup_Simplex.mac

## Set up verbosity
/control/execute verboseSetup.mac

## Invoke the application
/run/beamOn 10
```

バッチ処理なので可視化は不必要 - VisSetup_Simplexマクロをコメントアウト

[注] これをしないでバッチを走らせると、OpenGLの画面が表示される

← 実害はないがperformanceは極端に劣化

event処理数を適当に設定する

3) アプリの引数に'GlobalSetup.mac'を与え実行して、端末にアプリの出力が出ることを確認

```
$ ../bin/Application_Main GlobalSetup.mac
```

← TestBenchでアプリ実行 (マクロ指定を忘れない)

4) redirectionを使うことでアプリの出力を端末でなく、ファイルに書き出せることを確認

```
$ ../bin/Application_Main GlobalSetup.mac > output.txt
```

[メモ] この手法は大量情報を書き出したい時に便利

[重要注意] 本課題を終了した後は、必ずGlobalSetup.macの内容をもとに戻しておくこと

“.g4session”を使ったユーザ・インターフェイスの選択

課題:2 ユーザインターフェイスとしてQtではなく、tcshを使う方法を知る

- 対話型処理では今までQtを使用してきた - QtはGeant4でのデフォルトのユーザインターフェイス
- Geant4はQt以外に以下のユーザインターフェイスを用意している

■ tcsh, csh, gag,....

[注] Qtが使えない環境では、tcsh/cshを使うのが良い

- ユーザインターフェイスの切り替えは“.g4session”というファイルを使うことで簡単に実行できる

1) ユーザのホームディレクトリにある“.g4session”ファイルを好みのエディタで開き、内容を以下の様に変更

```
g4vm: code ~/.g4session
```

```
# - .g4session
#
#tcsh
qt
```

tcshのコメントを外し、
qtをコメントアウトする

tcshに切り変わった

```
tcsh
#qt
```

2) TestBenchディレクトリに移動し、アプリを実行すると 端末にtschスタイルの対話処理セッションが開く

```
$ cd TestBench
$ ../bin/Application_Main
```

3) 対話処理セッションはユーザからのコマンド入力待ち 状態(Idle>)なので、以下のコマンドを入れてみる

```
Idle> /control/execute GlobalSetup.mac
```

別窓に可視化画面が開き、boxが表示されることを確認

4) 以下のコマンドを入れて粒子飛跡が見えることを確認

```
Idle> /run/beamOn 1
```

次ページのスライドを参照

```
*****
Geant4 version Name: geant4-11-03 [MT] (6-December-2024)
Copyright : Geant4 Collaboration
References : NIM A 506 (2003), 250-303
             : IEEE-TNS 53 (2006), 270-278
             : NIM A 835 (2016), 186-225
WWW : http://geant4.org/
*****

<<< Geant4 Physics List simulation engine: FTFP_BERT
.....
Available UI session types: [ Qt, GAG, tcsh, csh ]
Idle>
```

一部表示省略

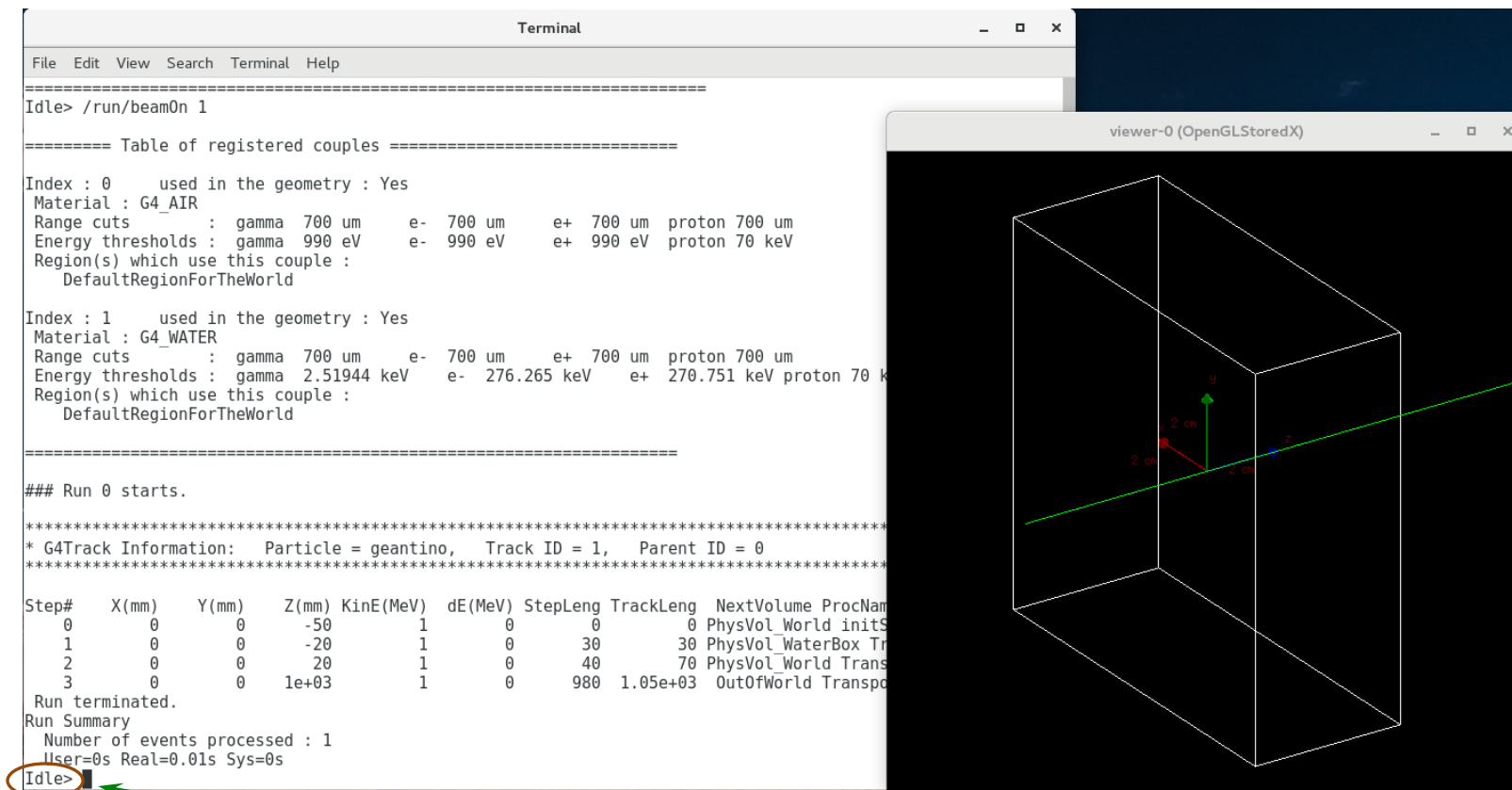
Qt環境の使用時にはQtがユーザのコマンドを待つ
ので、端末にはIdle>は表示されない

[重要注意]

本課題を終了した後は、必ず.g4sessionの内容をもと
に戻し、Qtを使うモードにしておくこと

“.g4session”を使ったユーザ・インターフェイスの選択 — つづき

ユーザインターフェイスをtcshして、アプリを走らせた例：



```
Terminal
File Edit View Search Terminal Help
=====
Idle> /run/beamOn 1

===== Table of registered couples =====
Index : 0      used in the geometry : Yes
Material : G4_AIR
Range cuts   : gamma 700 um   e- 700 um   e+ 700 um   proton 700 um
Energy thresholds : gamma 990 eV   e- 990 eV   e+ 990 eV   proton 70 keV
Region(s) which use this couple :
    DefaultRegionForTheWorld

Index : 1      used in the geometry : Yes
Material : G4_WATER
Range cuts   : gamma 700 um   e- 700 um   e+ 700 um   proton 700 um
Energy thresholds : gamma 2.51944 keV   e- 276.265 keV   e+ 270.751 keV   proton 70 keV
Region(s) which use this couple :
    DefaultRegionForTheWorld

=====

### Run 0 starts.

*****
* G4Track Information: Particle = geantino, Track ID = 1, Parent ID = 0
*****

Step#   X(mm)   Y(mm)   Z(mm)   KinE(MeV)   dE(MeV)   StepLeng   TrackLeng   NextVolume   ProcName
0       0       0       -50     1           0           0           0   PhysVol_World   initS
1       0       0       -20     1           0           30          30   PhysVol_WaterBox   Tr
2       0       0       20      1           0           40          70   PhysVol_World   Trans
3       0       0       1e+03   1           0          980       1.05e+03   OutOfWorld   Transp

Run terminated.
Run Summary
Number of events processed : 1
User=0s Real=0.01s Sys=0s
Idle>
```

[注 1] 端末プロンプトが'Idle>'と表示されていれば、ユーザコマンドの入力待ち状態になっている

[注 2] 表示画面の操作(回転、拡大など)は、visualization関連のユーザコマンドを入力する